

#### **4\_301: Testing in Django**

```
# pip install factory_boy==3.0.1
```

#### **4\_302: Factory Boy**

##### **model\_factories.py**

```
import factory

from django.test import TestCase
from django.conf import settings
from django.core.files import File

from .models import *

class ECFactory(factory.django.DjangoModelFactory):
    ec_name = "transferase"

    class Meta:
        model = EC

class SequencingFactory(factory.django.DjangoModelFactory):
    sequencing_factory = "Sanger"
    factory_location = "UK"

    class Meta:
        model = Sequencing

class GeneFactory(factory.django.DjangoModelFactory):
    gene_id = "GeneX"
    entity = "Plasmid"
    start = 12
    stop = 100
    sense = "+"
    start_codon = "M"
    sequencing = factory.SubFactory(SequencingFactory)
    ec = factory.SubFactory(ECFactory)
    access = 0

    class Meta:
        model = Gene
```

#### **4\_303: Writing API test**

##### **urls.py**

```
path('api/genes', views.GeneList.as_view(),
name='genes_api'),
path('api/gene/<int:pk>/', api.GeneDetail.as_view(),
name='gene_api'),
```

##### **tests.py**

```
import json
```

```

from django.test import TestCase
from django.urls import reverse
from django.urls import reverse_lazy

from rest_framework.test import APIRequestFactory
from rest_framework.test import APITestCase

from .model_factories import *
from .serializers import *

class GeneTest(APITestCase):

    def test_geneDetailReturnsSuccess(self):
        gene = GeneFactory.create(pk=1, gene_id="gene1")
        url = reverse('gene_api', kwargs={'pk': 1})
        response = self.client.get(url)
        response.render()
        self.assertEqual(response.status_code, 200)

    def test_geneDetailReturnFailOnBadPk(self):
        gene = GeneFactory.create(pk=2, gene_id="gene2")
        url = "/api/gene/H/"
        response = self.client.get(url)
        self.assertEqual(response.status_code, 404)

```

## tests.py

```

gene1 = None
gene2 = None
good_url = ''
bad_url = ''

def setUp(self):
    self.gene1 = GeneFactory.create(pk=1, gene_id="gene1")
    self.gene1 = GeneFactory.create(pk=2, gene_id="gene2")
    self.good_url = reverse('gene_api', kwargs={'pk': 1})
    self.bad_url = "/api/gene/H/"

def tearDown(self):
    EC.objects.all().delete()
    Sequencing.objects.all().delete()
    Gene.objects.all().delete()
    ECFactory.reset_sequence(0)
    SequencingFactory.reset_sequence(0)
    GeneFactory.reset_sequence(0)

def test_geneDetailReturnsSuccess(self):
    response = self.client.get(self.good_url, format='json')
    response.render()
    self.assertEqual(response.status_code, 200)
    data = json.loads(response.content)
    self.assertTrue('entity' in data)
    self.assertEqual(data['entity'], 'Plasmid')

```

```
def test_geneDetailReturnFailOnBadPk(self):
    response = self.client.get(self.bad_url, format='json')
    self.assertEqual(response.status_code, 404)
```

tests.py

```
delete_url = ''

self.gene3 = GeneFactory.create(pk=3, gene_id="gene3")
self.delete_url = reverse('gene_api', kwargs={'pk': 3})

def test_geneDetailDeleteIsSuccessful(self):
    response = self.client.delete(self.delete_url,
format='json')
    self.assertEqual(response.status_code, 204)
```

#### 4.304 Serialiser tests

**tests.py**

```
from .serializers import *
```

```
class GeneSerialiserTest(APITestCase):
    gene1 = None
    geneserializer = None

    def setUp(self):
        self.gene1 = GeneFactory.create(pk=1, gene_id="gene1")
        self.geneserializer = GeneSerializer(instance=self.gene1)

    def tearDown(self):
        EC.objects.all().delete()
        Sequencing.objects.all().delete()
        Gene.objects.all().delete()
        ECFactory.reset_sequence(0)
        SequencingFactory.reset_sequence(0)
        GeneFactory.reset_sequence(0)

    def test_geneSerilaiserHasCorrectFields(self):
        data = self.geneserializer.data
        self.assertEqual(set(data.keys()), set(['gene_id',
'sequencing',
'sense', 'start',
'stop',
'entity', 'ec',
'start_codon']))

    def test_geneSerilaiserGeneIDIsHasCorrectData(self):
        data = self.geneserializer.data
        self.assertEqual(data['gene_id'], "gene1")
```

**model\_factories.py**

```
from random import randint
```

```
class GeneFactory(factory.django.DjangoModelFactory):
    gene_id = factory.Sequence(lambda n: 'gene%d' % n+str(1))
    entity = factory.Faker('sentence', nb_words=1)
    start = randint(1, 100000)
    stop = start+randint(1, 10000)
```

#### **model\_factories.py**

```
from random import choice
entity = choice(['Plasmid', 'Chromosome'])
```